# Experiment No. 6

## Aim: Implement authentication and user roles with JWT

### Source Code:

WeatherDashboard.jsx

```jsx
import React, { useEffect, useState } from 'react';
import axios from 'axios';
import { useNavigate } from 'react-router-dom';
import moment from 'moment';

const WeatherDashboard = () => {
  const [city, setCity] = useState('');
  const [weatherCards, setWeatherCards] = useState([]);
  const [currentLocationWeather, setCurrentLocationWeather] = useState(null);
  const [currentLocationForecast, setCurrentLocationForecast] = useState([]);
  const [error, setError] = useState('');
  const [user, setUser] = useState(null);
  const [history, setHistory] = useState(() => JSON.parse(localStorage.getItem("history")) || []);
  const navigate = useNavigate();

  const API_KEY = '1b1025147cab66d0fc1c78fb04624148';

  const getBackgroundImage = (weatherCondition) => {
    const condition = weatherCondition?.toLowerCase();
    if (condition?.includes('clear') || condition?.includes('sunny')) return 'https://images.unsplash.com/photo-1517441295968-38501e56b3e6?crop=entropy&cs=t
    if (condition?.includes('cloud') || condition?.includes('overcast')) return 'https://images.unsplash.com/photo-1507525428034-b723cf961dde?crop=entropy&c
    if (condition?.includes('rain') || condition?.includes('drizzle')) return 'https://images.unsplash.com/photo-1534082723326-0e104e76d912?crop=entropy&cs=
    if (condition?.includes('storm') || condition?.includes('thunder')) return 'https://images.unsplash.com/photo-1507663260840-7e5d8ac8545e?crop=entropy&cs
    if (condition?.includes('snow')) return 'https://images.unsplash.com/photo-1549729598-c11ce4465492?crop=entropy&cs=tinysrgb&fit=max&fm=jpg&ixid=M3w0NTQz
    if (condition?.includes('haze') || condition?.includes('mist') || condition?.includes('fog')) return 'https://images.unsplash.com/photo-1514751110006-2f
    return 'https://images.unsplash.com/photo-1503389025263-e3801f5c7112?crop=entropy&cs=tinysrgb&fit=max&fm=jpg&ixid=M3w0NTQzMjB8MHwxfHNlYXJjaHw1fHxvY2Vhbi
  };

  const fetchCurrentLocationWeather = async (lat, lon) => {
    try {
      const res = await axios.get(`https://api.openweathermap.org/data/2.5/weather?lat=${lat}&lon=${lon}&appid=${API_KEY}&units=metric`);
      setCurrentLocationWeather(res.data);
      const forecastRes = await axios.get(`https://api.openweathermap.org/data/3.0/onecall?lat=${lat}&lon=${lon}&exclude=minutely,hourly,alerts&appid=${API_
      setCurrentLocationForecast(forecastRes.data.daily.slice(0, 7));
    } catch (err) {
      setError('Could not fetch your location's weather.');
    }
  };
```

```jsx
                </div>
                <p className="text-3xl font-extrabold text-indigo-600">{Math.round(weather.main.temp)}°C</p>
                <p className="capitalize text-gray-500">{weather.weather[0].description}</p>
                <div className="mt-4 grid grid-cols-2 sm:grid-cols-3 md:grid-cols-5 gap-2 text-xs text-center text-gray-700">
                  {weather.forecast.map((f, i) => (
                    <div key={i} className="bg-gray-100 p-2 rounded-lg">
                      <p className="font-medium">{moment(f.dt_txt).format('ddd')}</p>
                      <img src={getWeatherIcon(f.weather[0].icon)} alt="Forecast icon" className="w-10 h-10 mx-auto" />
                      <p className="font-bold text-indigo-700">{Math.round(f.main.temp)}°C</p>
                    </div>
                  ))}
                </div>
              </div>
            ))}
          </div>
        )}

        {/* History Section */}
        {history.length > 0 && (
          <div className="mt-auto pt-6 border-t-2 border-gray-300">
            <div className="flex justify-between items-center text-sm text-gray-600 mb-2">
              <span>Recent searches:</span>
              <button onClick={clearHistory} className="text-red-500 hover:text-red-700 underline transition">Clear</button>
            </div>
            <div className="flex flex-wrap gap-2">
              {history.map((h, i) => (
                <span key={i} className="bg-gray-300 px-3 py-1 rounded-full text-gray-700 text-sm font-medium">{h}</span>
              ))}
            </div>
          </div>
        )}
      </div>
    </div>
  );
};

export default WeatherDashboard;
```

AdminDashboard.jsx

```jsx
import axios from 'axios';
import React, { useEffect, useState } from 'react';
import { useNavigate } from 'react-router-dom';


const AdminDashboard = () => {
  const [users, setUsers] = useState([]);
  const navigate = useNavigate()
  const [isAdmin, setAdmin] = useState(false);

  const fetchDashboard = async () => {
    const token = localStorage.getItem("token");

    try {
      const res = await axios.get("http://localhost:5000/adminDashboard", {
        headers: { Authorization: `Bearer ${token}` }
      });
      setUsers(res.data.users);

      if (res.data.admin.role !== "admin") {
        navigate("/weatherDashboard");
      }
      navigate("/adminDashboard")
    } catch (err) {
      console.log("Access denied", err);
      navigate("/weatherDashboard"); // optional: redirect if not admin
    }
  };

  useEffect(() => {
    fetchDashboard();
  }, []);

  return (
    <div className="min-h-screen bg-gradient-to-r from-blue-100 via-purple-100 to-pink-100 px-4 p-6">
      <div className="max-w-5xl mx-auto">
        {/* Dashboard Header */}
        <div className="mb-8">
          <h1 className="text-3xl font-bold text-indigo-700 mb-2">Admin Dashboard</h1>
          <p className="text-gray-600">Overview of logged-in users</p>
        </div>

        {/* Stats Box */}
        <div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 gap-4 mb-6">
          <div className="bg-white shadow rounded-2xl p-6 text-center">
            <h2 className="text-xl font-semibold text-gray-800">Total Logged-in Users</h2>
            <p className="text-4xl text-indigo-600 mt-2">{users.length}</p>
          </div>
        </div>

        {/* User Table */}
        <div className="bg-white shadow-md rounded-2xl overflow-x-auto">
          <table className="min-w-full table-auto text-sm text-gray-700">
            <thead className="bg-indigo-50">
              <tr>
                <th className="px-6 py-3 text-left font-semibold">#</th>
                <th className="px-6 py-3 text-left font-semibold">Name</th>
                <th className="px-6 py-3 text-left font-semibold">Email</th>
                <th className="px-6 py-3 text-left font-semibold">Role</th>
                <th className="px-6 py-3 text-left font-semibold">Login Time</th>
              </tr>
            </thead>
            <tbody>
              {users.map((user, idx) => (
                <tr key={user.id} className="border-b hover:bg-gray-50">
                  <td className="px-6 py-3">{idx + 1}</td>
                  <td className="px-6 py-3">{user.name}</td>
                  <td className="px-6 py-3">{user.email}</td>
                  <td className="px-6 py-3">{user.role}</td>
                  <td className="px-6 py-3">{user.lastLogin}</td>
                </tr>
              ))}
            </tbody>
          </table>
        </div>
      </div>
```

```
    </div>
  );
};

export default AdminDashboard;
```

authMiddleware.jsx

```
const jwt = require("jsonwebtoken");
const SECRET = "ABCD@1234";


function authMiddleware(req,res,next){
    const authHeader = req.headers.authorization;
    if (!authHeader) return res.status(401).json({ message: "No token provided" });

    const token = authHeader.split(" ")[1]; // "Bearer <token>"

    jwt.verify(token, SECRET, (err, user) => {
      if (err) return res.status(403).json({ message: "Invalid token" });
      req.user = user;
      next();
    });

}

module.exports = authMiddleware;
```

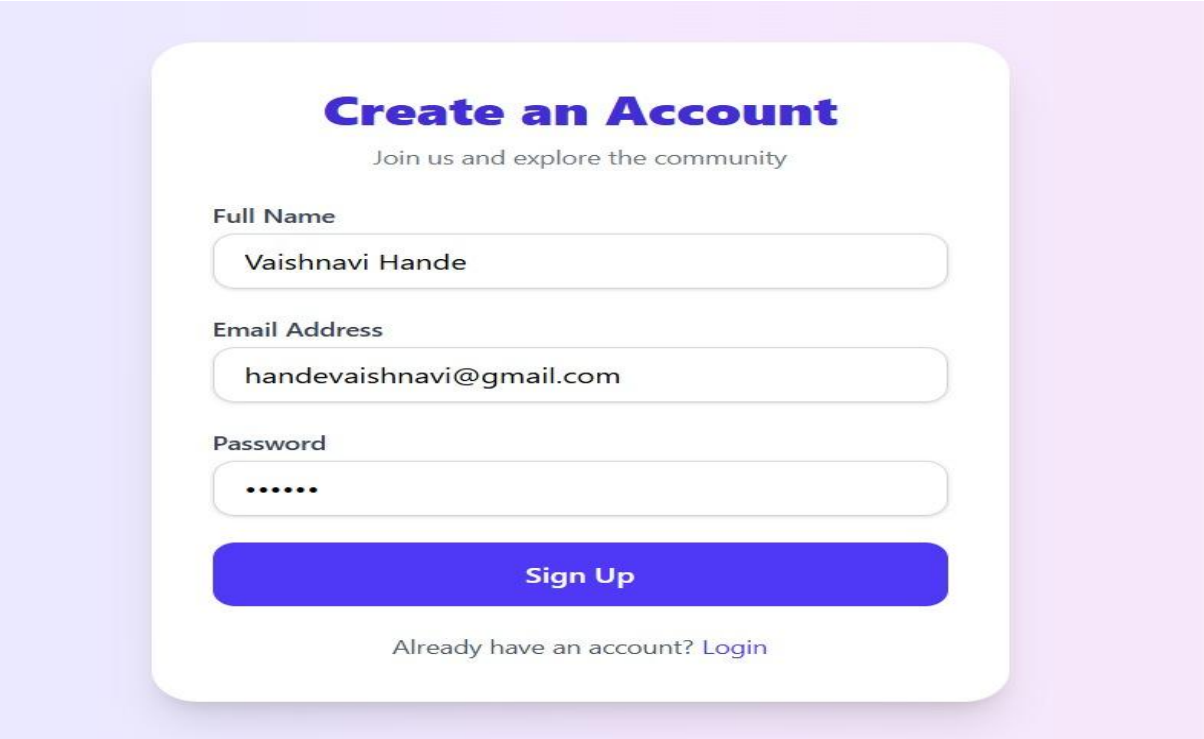
Index.jsx

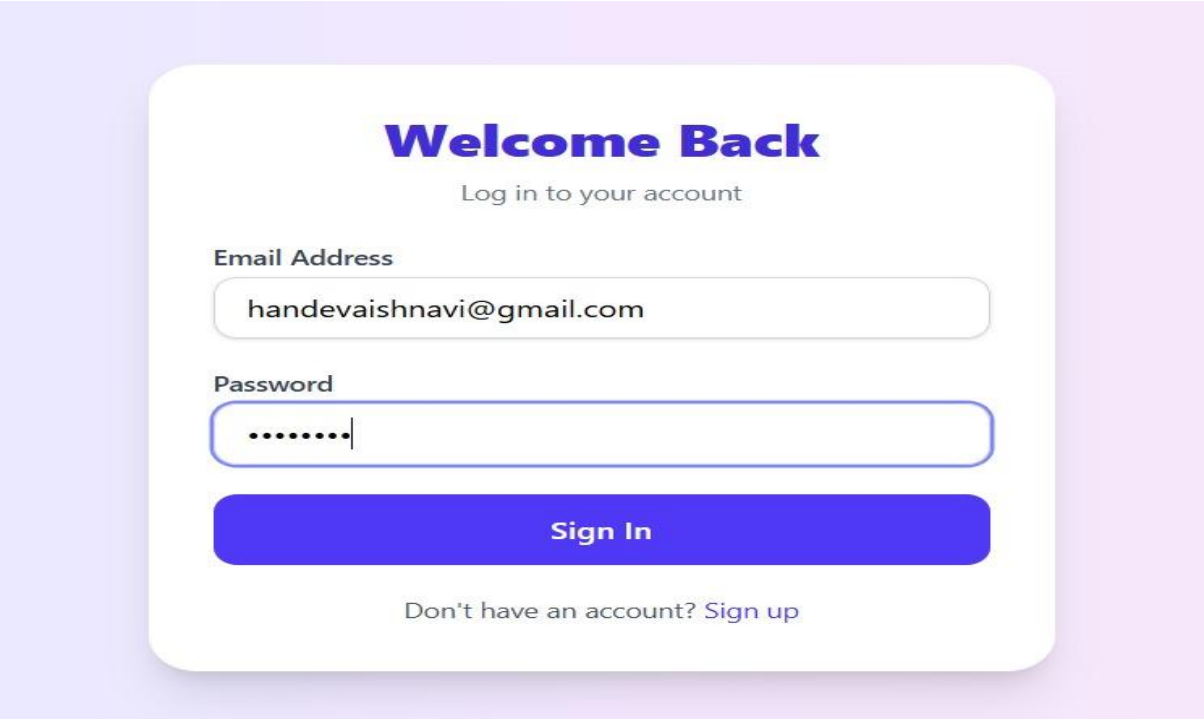
```
require("dotenv").config();  // ✅ Load .env first
const express = require('express');
const cors = require("cors");
const app = express();
const mongoose = require("mongoose");
const jwt = require("jsonwebtoken");
const bcrypt = require("bcryptjs");
const authenticateToken = require('./middleware/authMiddleware');
const authorizeRole = require('./middleware/authorizeRole');

// ✅ Use environment variables
const PORT = process.env.PORT || 5000;
const SECRET = process.env.JWT_SECRET || "ABCD@1234";
const MONGO_URI = process.env.MONGO_URI;

// ✅ Connect to MongoDB Atlas
mongoose.connect(MONGO_URI)
  .then(() => console.log("✅ Connected to MongoDB Atlas"))
  .catch(err => console.error("❌ MongoDB connection error:", err));

// ✅ User Schema & Model
const userSchema = new mongoose.Schema({
  name: String,
  email: { type: String, unique: true },
  hashedPass: String,
  role: { type: String, default: "user" },
  lastLogin: String
});

const User = mongoose.model("User", userSchema);

// ✅ Middleware
app.use(express.urlencoded({ extended: true }));
app.use(express.json());

var corsOptions = {
  origin: 'http://localhost:3000',
  optionsSuccessStatus: 200,
```

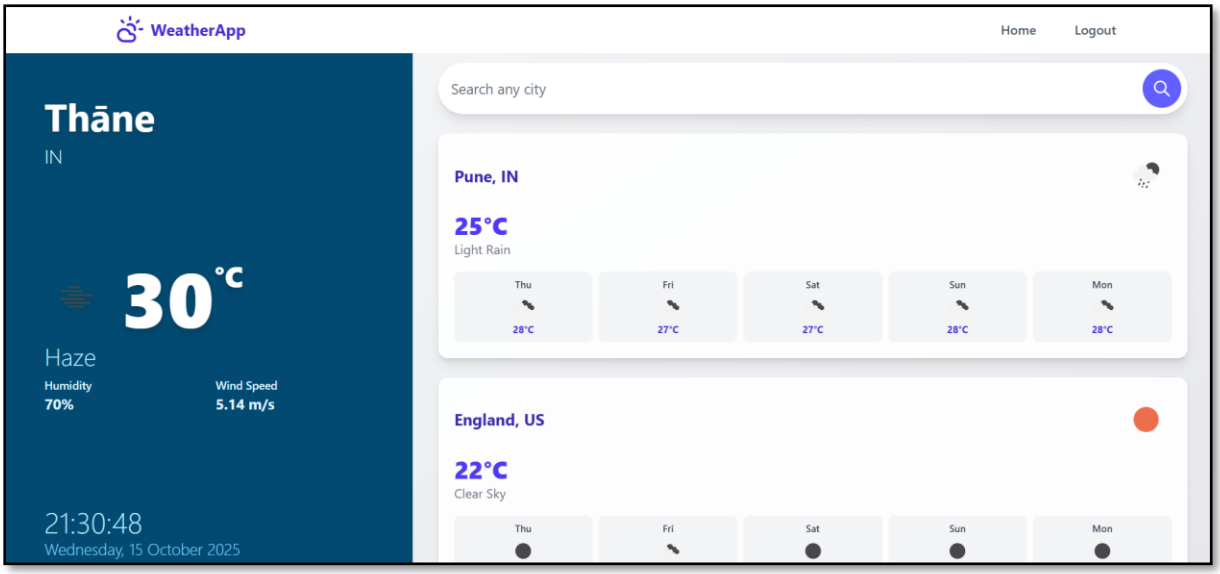
```
  credentials: true,
};
app.use(cors(corsOptions));

// ✅ SIGNUP
app.post("/signup", async (req, res) => {
  try {
    const { name, email, password, role } = req.body;

    const existingUser = await User.findOne({ email });
    if (existingUser) {
      return res.status(400).json({ message: "Email already registered", status: false });
    }

    const saltRounds = 10;
    const hashedPass = await bcrypt.hash(password, saltRounds);

    const newUser = new User({
      name,
      email,
      hashedPass,
      role: role || "user"
    });

    await newUser.save();
    res.status(201).json({ message: "User created", status: true });

  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

// ✅ LOGIN
app.post("/login", async (req, res) => {
  try {
    const { email, password } = req.body;

    const user = await User.findOne({ email });
```

```
    const isMatch = await bcrypt.compare(password, user.hashedPass);
    if (!isMatch) {
      return res.status(401).json({ message: "Invalid credentials", status: false });
    }

    // Update last login time
    user.lastLogin = new Date().toISOString();
    await user.save();

    // Generate JWT
    const token = jwt.sign({ id: user._id, role: user.role }, SECRET, { expiresIn: "1d" });

    res.status(200).json({
      message: "Login successful",
      token,
      role: user.role,
      lastLogin: user.lastLogin,
      status: true
    });

  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// ✅ DASHBOARD
app.get("/dashboard", authenticateToken, async (req, res) => {
  try {
    const user = await User.findById(req.user.id);
    if (!user) {
      return res.status(404).json({ message: "User not exists" });
    }
    res.json({ message: "Welcome to dashboard", user });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});
```

**Output:**


Create an Account
Join us and explore the community
Full Name
Vaishnavi Hande
Email Address
handevaishnavi@gmail.com
Password
Sign Up
Already have an account? Login


Welcome Back
Log in to your account
Email Address
handevaishnavi@gmail.com
Password
Sign In
Don't have an account? Sign up




## Conclusion:

I have successfully executed and implemented authentication and user roles with **JWT.**